

COP 3330 Midterm Exam Study Guide

Hotel Room Management System

A small hotel needs a simple program to keep track of its rooms: which rooms exist, what type they are, and whether each one is currently occupied. You will build two classes (Room and RoomTester) that work together to model this system.

Part 1 — The Room Class

Each Room object represents a single hotel room. A room has the following information, none of which should be directly accessible from outside the class:

- A room number (an integer).
- A room type, which must always be one of exactly three values: "single", "double", or "suite".
- An occupied status (true if a guest is currently checked in, false otherwise).

Constructors

- A no-argument constructor that builds a default room: room number 1, type "single", and not occupied.
- A three-argument constructor that accepts a room number, a room type, and an occupied status, and builds the room from those values.

Room numbers should be set using the class's own setter internally rather than being assigned directly, since this keeps the door open for validation later.

Methods

- `getRoomNumber()` returns the room's number.
- Create a method which will return true if given number is positive, less than 1000 and if it is less than 20 percent of the hundred percentage. Set the method name as *isValidRoomNo* and it will take an int as parameter. Ex: $147 \Rightarrow 147 / 100 = 1 \Rightarrow$ Check if $147 < 1 \times 100 + 20$. 147 is not less than 120. Therefore 147 will return false. For $317 \Rightarrow 317 / 100 \Rightarrow$ Check if 317 is less than $3 \times 100 + 20$, it will return true. This method will be used in room number verification because only 20 rooms can be allocated for one floor. Hotel building has maximum 20 rooms available for each floor. This method will be used in verification of room number. Note that this method does not touch the instance variables of this class.
- The ability to change a room's number should exist, but only code inside the Room class itself should ever be allowed to call it — outside code should never be able to change a room's number once the room exists. This method will call the *isValidRoomNo* method. If the set value is not valid, this method will print an error and set the room number to 999.
- `isOccupied()` returns whether the room currently has a guest.
- `checkin()` marks the room as occupied, but only if it isn't already occupied.
- `checkout()` marks the room as vacant.

- `setRoomType(String)` changes the room's type, but only if the given value is exactly "single", "double", or "suite". If an invalid value is passed, the room type must not change, and the program should print: Invalid room type entered
- `getRoomType()` returns the room's current type.
- `toString()` returns a description in the form: number 101 is occupied or number 101 is vacant
- `showRoom()` prints [x] if the room is occupied, or [] if it is vacant, with no line break afterward.

The system also needs to keep a running total of how many Room objects have been created since the program started — counting every room ever built through either constructor, not just the rooms that happen to still be in a particular list. There should be a way for outside code to ask for this total at any time.

Part 2 — The RoomTester Class

This class builds a small hotel out of Room objects and demonstrates that the Room class works correctly.

Methods

- `randomRoom(boolean validRoom)`: when *validRoom* is true, creates and returns a new Room with a randomly generated room number between 100 and 999 (inclusive of 100, exclusive of 1000), room type "single", and not occupied. Call *isValidRoomNo* to check if the generated room number is valid. If not create another random number. Do this until you get a valid room number. When *validRoom* is false, returns null instead of creating a room.
- `printAll(ArrayList<Room> list)`: prints every entry in the list, one per line. If an entry actually holds a room, print that room's own text description; if an entry is empty (no room was ever stored there), print the word null instead.
- `printHotel(ArrayList<Room> list)`: goes through the list and, for every entry that actually holds a room, prints that room's own bracket-style occupancy display (see `showRoom` above), all on the same line. Empty entries are skipped entirely — they contribute nothing to the line. A single line break is printed once the whole list has been shown.

Main Method

Write a main method that does the following, in order:

1. Create an empty ArrayList that will hold the hotel's rooms. Name the ArrayList as hotel.
2. Add four randomly generated rooms to the list using `randomRoom(true)` method you have implemented.
3. Call The `printAll()` method on your ArrayList.
4. Randomly create a boolean variable with roughly a half chance, and use that boolean variable as a parameter to call `randomRoom()` method. Add the created room to the ArrayList. Note that if the `randomRoom()` returns null, null will be added to the ArrayList, not Room object.

5. Print a message reporting the total number of rooms created so far, in the form: "{number} rooms are created."
6. In the ArrayList find the last room that is not null, and change that room's type to "single".
7. Call the printHotel method on your ArrayList

Part 3

For this part, do the same things that you have done in part 2 but use an array instead. Create a RoomTesterArray class and implement the methods. Set the array length as 5.

Notes

- If we want to get user input we will need to write `Scanner scan = new Scanner(System.in);` also we will need to import `java.util.Scanner`
- If we want to use ArrayList, we need to import `java.util.ArrayList`.
- If we want to use Math class, we do not need to import anything.
Ex: `Math.sqrt(5.0)` or `Math.random()` will work without importing anything.
- If we want to use StringTokenizer, we will need to
`import java.util.StringTokenizer;`
- If we want to use Random class we will need to write
`Random random = new Random();` before calling `random.nextInt()` and we will need to import `java.util.Random;`
- We will need to write package statement at the beginning of the class. The import statements should be also written immediately after *package* statement.
- Always write comments in your code.
- The solution of the sample exam study guide is below.

//Room.java

```
package rooms;

public class Room {

    private int roomNumber;
    private String roomType; // single, double, suite
    private boolean occupied;

    private static int count = 0;
    // to keep track of created objects

    public Room() {
        this(1, "single", false); // this syntax is more clean
    }

    public Room(int rn, String givenRoomType, boolean occ) {
        setRoomNumber(rn); // use setters not assignment
        setRoomType(givenRoomType);
        occupied = occ;
        count++;
    }

    public int getRoomNumber() {
        return roomNumber;
    }

    //this method does not touch the instance variables
    // it is better if it stays as static
    public static boolean isValidRoomNo(int x){
        if(x>0 && x<1000){
            int val=x/100;
            if(val*100+20>x){
                return true;
            }
        }
    }
}
```

```

        }
    }
    return false;
}

// this method should be only callable by the class
private void setRoomNumber(int rn) {
    if(isValidRoomNo(rn)){
        roomNumber = rn;
    }else{
        System.out.println("Error in setting room
number");
        roomNumber = 999;
    }
}

public boolean isOccupied() {
    return occupied;
}

public void checkin() {
    if (!isOccupied()) {
        occupied = true;
    }
}

public void checkout() {
    occupied = false;
}

public void setRoomType(String givenRoom) {
    if (givenRoom.equals("single") ||
        givenRoom.equals("double") ||

```

```

        givenRoom.equals("suite")) {
            roomType = givenRoom;
        } else {
            System.out.println("Invalid room type entered");
        }
    }

    public String getRoomType() {
        return roomType;
    }

    public String toString() {
        String result = "number " + roomNumber;
        if (isOccupied()){
            result += " is occupied";
        }else{
            result += " is vacant";
        }
        return result;
    }

    public void showRoom() {
        String result = "[ ]"; // assume the room is vacant
        if (isOccupied())
            result = "[x]"; // if occupied change the output
        System.out.print(result);
    }

    public static int getCount() {
        // since count is static this method should be static
        return count;
    }
} // end of class

```

```

//RoomTester.java
package rooms;

import java.util.ArrayList;
import java.util.Random;

public class RoomTester {

    private static Random rand = new Random();

    public static Room randomRoom(boolean validRoom) {
        Room r = null;
        if (validRoom) {
            int rnum = rand.nextInt(1000 - 100) + 100;
            /*try a random number until we get a good one.
            isValidRoomNo is a static method. We can call from outside
            as long as it is not private */
            while(Room.isValidRoomNo(rnum) == false){
                rnum = rand.nextInt(1000 - 100) + 100;
            }
            r = new Room(rnum, "single", false);
        }
        return r;
    }

    public static void printAll(ArrayList<Room> arr) {
        for (int i = 0; i < arr.size(); i++) {
            if(arr.get(i)!=null){
                System.out.println(arr.get(i));
            }else{
                System.out.println("null");
            }
        }
    }
}

```

```
}
```

```
public static void printHotel(ArrayList<Room> arr) {  
    for (int i = 0; i < arr.size(); i++) {  
        if(arr.get(i)!=null){  
            arr.get(i).showRoom();  
        }  
    }  
    System.out.println();  
}
```

```
public static void main(String[] args) {  
  
    ArrayList<Room> hotel = new ArrayList<Room>();  
  
    for (int i = 0; i < 4; i++) {  
        hotel.add(randomRoom(true));  
    }  
  
    printAll(hotel);  
  
    boolean randbool = false;  
    if (rand.nextInt(6) > 3) {  
        randbool = true;  
    }  
  
    hotel.add(randomRoom(randbool));  
  
    System.out.println(Room.getCount() + " rooms are  
created");  
  
    boolean done = false;  
    for (int i = hotel.size() - 1; i >= 0 && !done; i--) {
```

```
        if (hotel.get(i) != null) {
            hotel.get(i).setRoomType("single");
            done = true;
        }
    }

    printHotel(hotel);

} // end of main

} // end of class
```

//RoomTesterArray.java

```

package rooms;
import java.util.Random;

public class RoomTesterArray {

    private static Random rand = new Random();

    public static Room randomRoom(boolean validRoom) {
        Room r = null;
        if (validRoom) {
            int rnum = rand.nextInt(1000 - 100) + 100;
            /*try a random number until we get a good one.
isValidRoomNo is a static method. We can call from outside,
as long as it is not private */
            while(Room.isValidRoomNo(rnum) == false){
                rnum = rand.nextInt(1000 - 100) + 100;
            }
            r = new Room(rnum, "single", false);
        }
        return r;
    }

    public static void printAll(Room[] arr) {
        for (Room myRoom:arr) { //we will only read the array
            /*similar to ArrayLists arrays can be traversed with
enhanced for loop*/
            if(myRoom!=null){
                System.out.println(myRoom);
            }else{
                System.out.println("null");
            }
        }
    }
}

```

```

public static void printHotel(Room[] arr) {
    for (int i = 0; i < arr.length; i++) {
        if(arr[i]!=null){
            arr[i].showRoom();
        }
    }
    System.out.println();
}

public static void main(String[] args) {

    Room[] hotel = new Room[5];

    /* we are not using 4 as hardcoded here. We could have
    done in that way also but here we are trying to impose the reader
    that we are filling every spot until the last position. In this
    way the code is more descriptive. If the array size change,
    hardwritten number 4 will look weird. The reader need to guess
    why it is 4.*/
    for (int i = 0; i < hotel.length-1; i++) {
        hotel[i]= randomRoom(true);
    }

    printAll(hotel);

    boolean randbool = false;
    //we are flipping a coin here
    if (Math.random() > 0.5) {
        // Math.random creates a floating point number [0,1)
        randbool = true;
    }

    hotel[4] = randomRoom(randbool);

```

```
        System.out.println(Room.getCount() + " rooms are
created");

        boolean done = false;
        //reverse traverse
        for (int i = hotel.length - 1; i >= 0 && !done; i--) {
            if (hotel[i] != null) {
                hotel[i].setRoomType("single");
                done = true;
            }
        }

        printHotel(hotel);

    } // end of main

} // end of class
```